Звіт

до лабораторної роботи №2

Тема: Створення UI за допомогою Jetpack Compose

Студент: AI-234

ПІБ: Сіголаєв Олександр

Посилання на GitHub - https://github.com/sigolaevoleks/Sigolaev_AI234_Lab2

Мета роботи

Метою лабораторної роботи є вивчення основ побудови графічного інтерфейсу користувача за допомогою фреймворку Jetpack Compose у середовищі Android Studio, ознайомлення з архітектурним підходом ViewModel + StateFlow, засвоєння принципів дизайну Material 3 та реалізація інтерактивного додатку згідно з індивідуальним варіантом.

Завдання роботи

Відповідно до варіанту №5 «Нотатки з розгортанням» необхідно було створити Android-додаток з використанням Jetpack Compose, який реалізує функціонал нотаток із можливістю розгортання та згортання карток. Під час виконання роботи було реалізовано наступні функції:

- створення та редагування нотаток через ModalBottomSheet;
- розгортання та згортання карток нотаток;
- пошук та фільтрація нотаток;
- закріплення (pin) нотаток;
- видалення нотаток свайпом з можливістю скасування через Snackbar;
- вибір кольору нотатки з 8 тональних варіантів;
- копіювання тексту нотатки в буфер обміну;
- відносні мітки часу (Just now, 2m ago, 3h ago тощо);

- перетягування нотаток для зміни порядку (drag-to-reorder);
- порожні стани для відсутності нотаток та відсутності результатів пошуку.

Хід роботи

Під час виконання лабораторної роботи було створено Android-проєкт у середовищі Android Studio з використанням Jetpack Compose та Material 3. Архітектура додатку побудована на патерні ViewModel з реактивними потоками StateFlow. Розглянемо основні компоненти реалізації.

1. Модель даних

Модель Note реалізована як data class з полями для ідентифікатора, заголовку, тексту, часу створення, стану закріплення та індексу кольору.

```
data class Note(
    val id: Int,
    val title: String,
    val text: String,
    val createdAt: Long = System.currentTimeMillis(),
    val isPinned: Boolean = false,
    val colorIndex: Int = 0
)
```

Використання Long замість форматованого рядка для часу дозволяє обчислювати відносні мітки часу (relative timestamps).

2. ViewModel

NoteViewModel керує станом додатку через MutableStateFlow та надає похідні потоки для UI. Фільтрація та сортування реалізовані через оператор combine.

```
class NoteViewModel : ViewModel() {

    private var nextId = 1
    private val _notes = MutableStateFlow<List<Note>>(emptyList())
    private val _expandedNoteId = MutableStateFlow<Int?>(null)
    private val _searchQuery = MutableStateFlow("")

    val notes: StateFlow<List<Note>> = combine(_notes, _searchQuery)
{ list, query ->
        val filtered = if (query.isBlank()) list else list.filter { note ->
```

```
            note.title.contains(query, ignoreCase = true) ||
                note.text.contains(query, ignoreCase = true)
        }
        filtered.sortedByDescending { it.isPinned }
    }.stateIn(viewModelScope, SharingStarted.Eagerly, emptyList())

    fun addNote(title: String, text: String, colorIndex: Int = 0) {
        val note = Note(id = nextId++, title = title, text = text,
            createdAt = System.currentTimeMillis(), colorIndex =
colorIndex)
        _notes.value = listOf(note) + _notes.value
    }

    fun updateNote(id: Int, title: String, text: String, colorIndex: Int) {
        _notes.value = _notes.value.map { note ->
            if (note.id == id) note.copy(title = title, text = text,
                colorIndex = colorIndex) else note
        }
    }

    fun deleteNote(id: Int): Note? {
        val deleted = _notes.value.find { it.id == id }
        _notes.value = _notes.value.filter { it.id != id }
        return deleted
    }

    fun restoreNote(note: Note) {
        _notes.value = _notes.value + note
    }

    fun moveNote(movedNoteId: Int, targetNoteId: Int) {
        val list = _notes.value.toMutableList()
        val movedIndex = list.indexOfFirst { it.id == movedNoteId }
        val targetIndex = list.indexOfFirst { it.id == targetNoteId }
        if (movedIndex == -1 || targetIndex == -1) return
        val note = list.removeAt(movedIndex)
        list.add(targetIndex, note)
        _notes.value = list
    }
}
```

Метод deleteNote() повертає видалену нотатку для реалізації скасування через Snackbar, а moveNote() забезпечує зміну порядку нотаток при перетягуванні.

3. Картка нотатки (NoteCard)

Компонент NoteCard реалізує відображення окремої нотатки з анімованим розгортанням, контекстним меню (⋮), drag-індикатором та підтримкою 8 тональних кольорів.

```
@Composable
fun NoteCard(
    note: Note,
    isExpanded: Boolean,
    onToggleExpanded: () -> Unit,
    onTogglePin: () -> Unit,
    onDelete: () -> Unit,
    onEdit: () -> Unit,
    onCopy: () -> Unit,
    dragHandleModifier: Modifier = Modifier
) {
    val containerColor = noteContainerColor(note.colorIndex, isExpanded)
    val contentColor = noteContentColor(note.colorIndex, isExpanded)

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .animateContentSize()
            .clickable(onClick = onToggleExpanded),
        colors = CardDefaults.cardColors(
            containerColor = containerColor,
            contentColor = contentColor
        )
    ) {
        // Drag handle + pin icon + relative timestamp + overflow menu
        // Title
        // Body (truncated to 50 chars when collapsed)
    }
}
```

Кожна картка має overflow-меню з опціями: Edit, Copy to clipboard, Pin/Unpin, Delete. Час відображається у відносному форматі через функцію formatRelativeTime().

4. Вибір кольору нотатки

Для підтримки кольорів нотаток реалізовано 8 варіантів з автоматичною адаптацією до світлої та темної теми. Серед них — 5 стандартних M3 контейнерних кольорів та 3 кастомних: зелений, синій, помаранчевий.

```
@Composable
private fun noteContainerColor(colorIndex: Int, isExpanded: Boolean): Color
{
    val dark = isSystemInDarkTheme()
    return when (colorIndex) {
        1 -> MaterialTheme.colorScheme.primaryContainer
        2 -> MaterialTheme.colorScheme.secondaryContainer
        3 -> MaterialTheme.colorScheme.tertiaryContainer
        4 -> MaterialTheme.colorScheme.errorContainer
        5 -> if (dark) GreenContainerDark else GreenContainerLight
        6 -> if (dark) BlueContainerDark else BlueContainerLight
        7 -> if (dark) OrangeContainerDark else OrangeContainerLight
        else -> if (isExpanded)
MaterialTheme.colorScheme.secondaryContainer
                else MaterialTheme.colorScheme.surfaceContainerLow
    }
}
```

У BottomSheet для вибору кольору реалізовано компонент ColorPicker з круглими зразками кольорів та іконкою ✓ для обраного.

5. Головний екран (NoteScreen)

NoteScreen об'єднує всі компоненти: TopAppBar з лічильником нотаток, пошуковий рядок, LazyColumn зі SwipeToDismissBox для кожної картки, порожні стани, FAB для створення нотатки та ModalBottomSheet для введення/редагування.

```
@Composable
fun NoteScreen(viewModel: NoteViewModel) {
    val notes by viewModel.notes.collectAsStateWithLifecycle()
```

```
    Scaffold(
        topBar = { TopAppBar(title = { Text("My Notes") }, ...) },
        floatingActionButton = {
            FloatingActionButton(onClick = { showBottomSheet = true }) {
                Icon(Icons.Default.Add, ...)
            }
        },
        snackbarHost = { SnackbarHost(snackbarHostState) }
    ) { innerPadding ->
        // Search bar
        // Empty states (no notes / no search results)
        // LazyColumn with SwipeToDismissBox + drag-to-reorder
    }
}
```

6. Swipe-to-delete з undo

Для видалення нотаток реалізовано свайп справа наліво через SwipeToDismissBox. Після видалення з'являється Snackbar з кнопкою «Undo», що дозволяє відновити нотатку.

```
fun handleDelete(noteId: Int) {
    val deleted = viewModel.deleteNote(noteId) ?: return
    scope.launch {
        val result = snackbarHostState.showSnackbar(
            message = deletedMessage,
            actionLabel = undoLabel,
            duration = SnackbarDuration.Short
        )
        if (result == SnackbarResult.ActionPerformed) {
            viewModel.restoreNote(deleted)
        }
    }
}
```

7. Drag-to-reorder

Зміна порядку нотаток реалізована через detectDragGesturesAfterLongPress на drag-індикаторі (⠿). Під час перетягування елемент піднімається з тінню

через graphicsLayer, а при перетині порогу 50% висоти елемента відбувається swap.

```
Modifier.pointerInput(note.id) {
    detectDragGesturesAfterLongPress(
        onDragStart = {
            draggedNoteId = note.id
            dragOffsetY = 0f
        },
        onDrag = { change, dragAmount ->
            change.consume()
            dragOffsetY += dragAmount.y
            // Swap logic when threshold crossed
        },
        onDragEnd = { draggedNoteId = null }
    )
}
```

Swap дозволяється лише між нотатками однієї групи (обидві закріплені або обидві незакріплені). Drag вимикається під час пошуку.

8. Копіювання в буфер обміну

Через пункт «Copy to clipboard» у контекстному меню реалізовано копіювання заголовку та тексту нотатки через Android ClipboardManager.

```
fun handleCopy(note: Note) {
    val clipboard = context.getSystemService(Context.CLIPBOARD_SERVICE)
        as ClipboardManager
    val clip = ClipData.newPlainText("note",
        "${note.title}\n${note.text}")
    clipboard.setPrimaryClip(clip)
    scope.launch {
        snackbarHostState.showSnackbar(
            copiedMessage, duration = SnackbarDuration.Short)
    }
}
```

9. Відносні мітки часу

Замість фіксованого часу створення відображається відносний час: «Just now», «2m ago», «3h ago», «5d ago» або конкретна дата для старіших нотаток.

```
private fun formatRelativeTime(createdAt: Long): String {
    val now = System.currentTimeMillis()
    val diffMs = now - createdAt
    val diffMinutes = diffMs / 1000 / 60
    val diffHours = diffMinutes / 60
    val diffDays = diffHours / 24

    return when {
        diffMs / 1000 < 60 -> "Just now"
        diffMinutes < 60 -> "${diffMinutes}m ago"
        diffHours < 24 -> "${diffHours}h ago"
        diffDays < 7 -> "${diffDays}d ago"
        else -> Instant.ofEpochMilli(createdAt)
            .atZone(ZoneId.systemDefault())
            .toLocalDate()
            .format(DateTimeFormatter.ofPattern("MMM d"))
    }
}
```

10. MainActivity

Головна активність ініціалізує ViewModel через делегат by viewModels() та встановлює тему з підтримкою dynamic color.

```
class MainActivity : ComponentActivity() {
    private val viewModel: NoteViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Sigolaev_AI234_Lab2Theme {
                NoteScreen(viewModel = viewModel)
            }
        }
    }
}
```

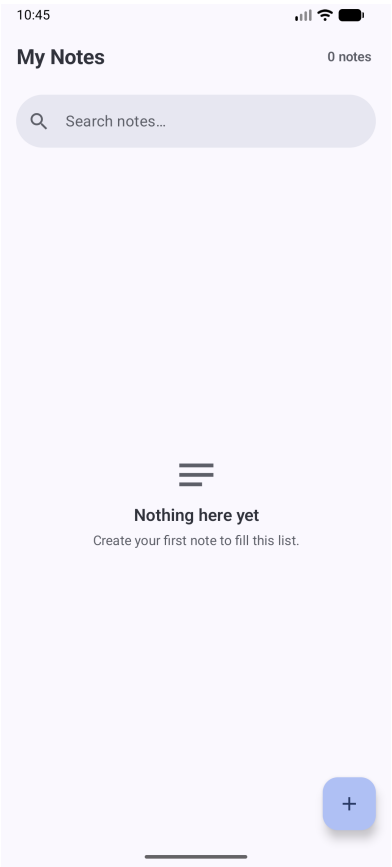
11. Тема додатку

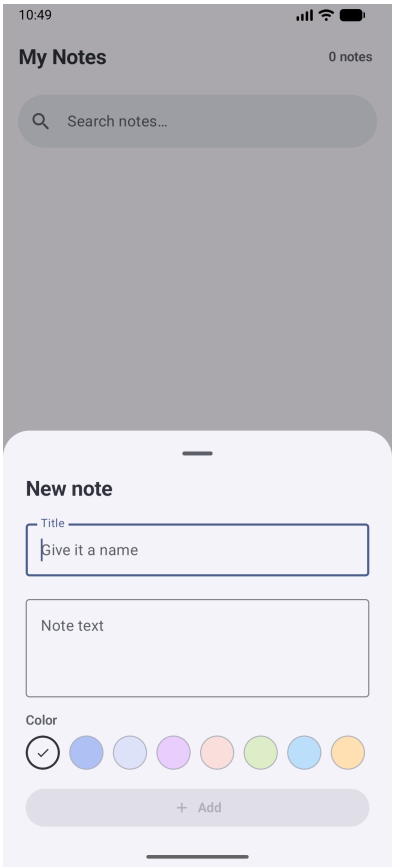
Тема реалізована з підтримкою Dynamic Color (Android 12+) та кастомною палітрою Evergreen/Terracotta/Gold як fallback.

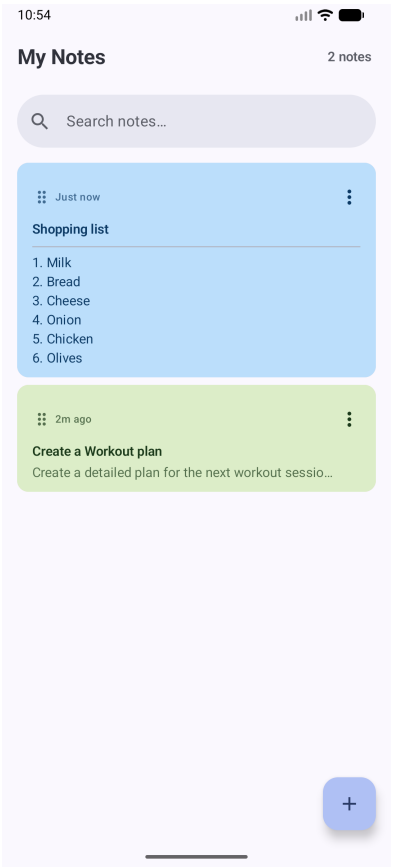
```
@Composable
fun Sigolaev_AI234_Lab2Theme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }
    MaterialTheme(colorScheme = colorScheme, content = content)
}
```

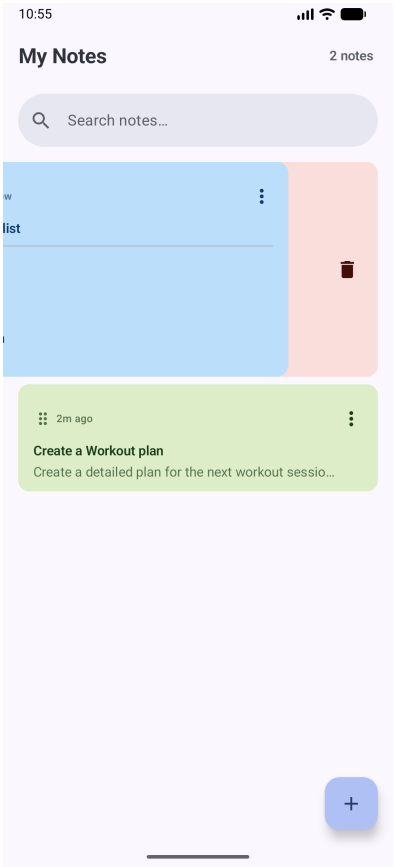
Скріншоти:

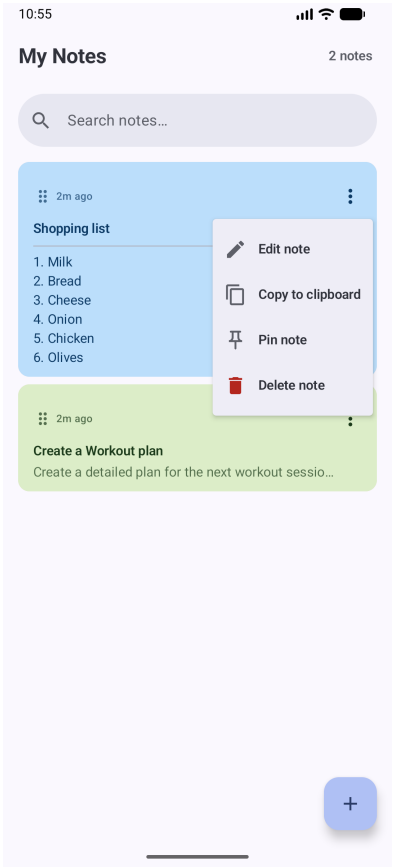
Порожній стан додатку (empty state); Список нотаток з різними кольорами; Розгорнута нотатка; BottomSheet створення/редагування з вибором кольору; Swipe-to-delete з Snackbar Undo; Пошук нотаток та інші …

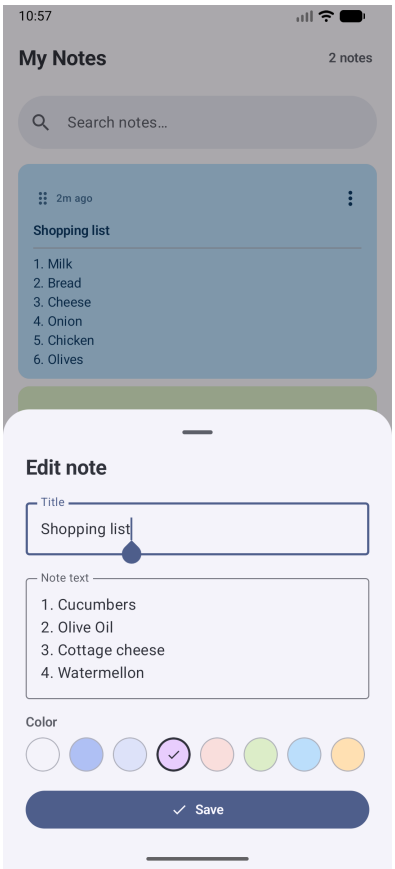


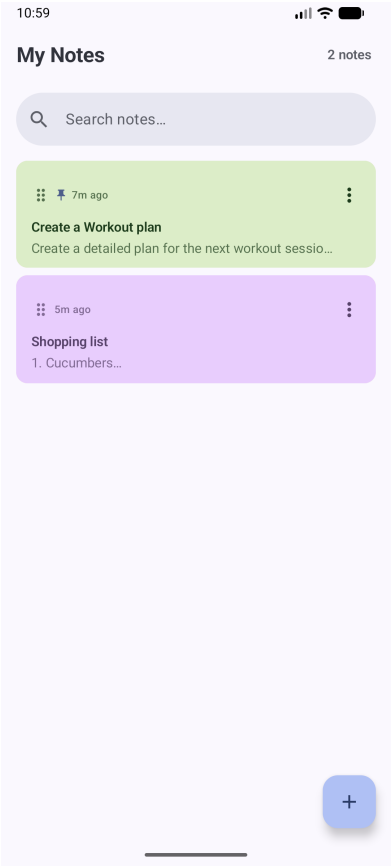


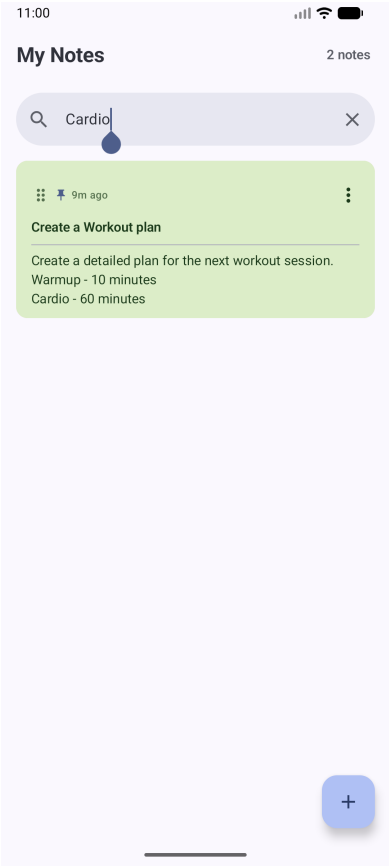


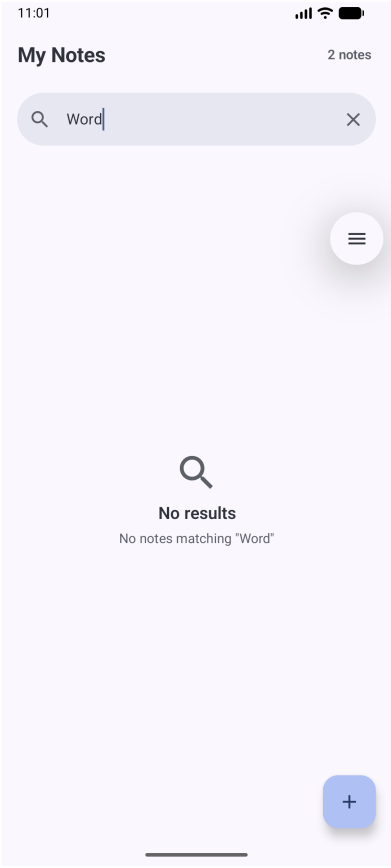












Результати виконання роботи

У результаті виконання лабораторної роботи було створено повнофункціональний Android-додаток «My Notes» з використанням Jetpack Compose та Material 3. Реалізовано архітектуру ViewModel + StateFlow для реактивного управління станом. Додаток підтримує створення, редагування, видалення та пошук нотаток, закріплення важливих нотаток, вибір кольору з 8 тональних варіантів, свайп-видалення зі скасуванням, перетягування для зміни порядку, копіювання в буфер обміну та відносні мітки часу.

Висновок

Під час виконання лабораторної роботи було успішно опрацьовано основи побудови графічного інтерфейсу за допомогою Jetpack Compose. Вивчено декларативний підхід до створення UI, принципи роботи з Material 3 компонентами (Card, Scaffold, TopAppBar, FloatingActionButton, ModalBottomSheet, SwipeToDismissBox, Snackbar), використання ViewModel та StateFlow для управління станом, жести перетягування через pointerInput та анімації через animateContentSize та graphicsLayer.

У результаті виконання лабораторної роботи були сформовані практичні навички створення інтерактивних Android-додатків з використанням сучасних інструментів та підходів до розробки мобільних інтерфейсів.